3.16 定点数的除法运算——原码的除法——不恢复余数法（加减交替法）

考纲定位：计算机组成原理 · 第 3 章 运算方法（3.16） | 重要度 ★★★★★ | 常见考法：大题（加减交替规则、上商、余数修正、与原码除法关系）

一句话核心：余数为负时不加回除数，而是下一步改“减”为“加”；这就是加减交替法

一、针对恢复余数过程的改进

当 R_i 为负数时，我们需要恢复余数，数学表达式如下



推导

- 恢复余数：$R_i + Y$
- 下一步左移后再减 Y：$2(R_i + Y) - Y = 2R_i + Y$
- 所以，余数为负时，不必恢复，直接左移后加 Y

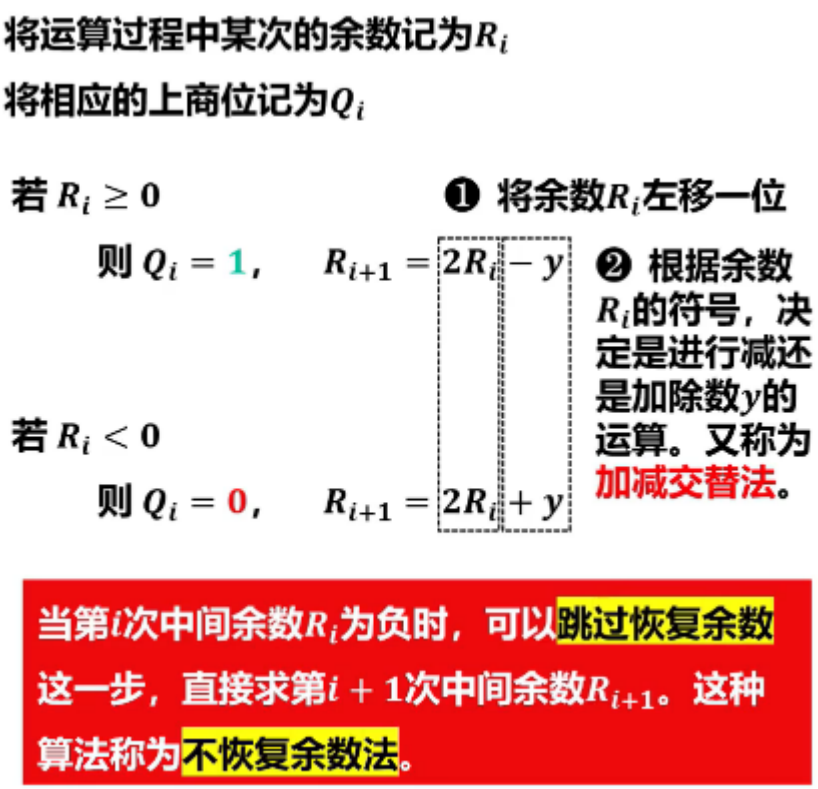
将 R_i 为负的表达式展开

若 $R_i<0$

则 $Q_i=0$， $R_{i+1}=2R_i+y$

当第i次中间余数R_i为负时，可以跳过恢复余数这一步，直接求第$i+1$次中间余数R_{i+1}。这种算法称为不恢复余数法。

总结



规则

- $R_i \geq 0$：商 1，下一步减 Y
- $R_i < 0$：商 0，下一步加 Y
- 每步先左移，再按上一步余数符号选择加/减
- 最后若余数为负，需要修正（加回 Y）

易错

- “加减交替”不是随意交替，而是由余数符号决定
- 余数为负时商 0，但下一步不是继续减，而是加
- 最后余数可能为负，需要恢复；商也可能需要修正

二、举例

【举例】设$[x]_{原}=1.1001$，$[y]_{原}=0.1011$，用不恢复余数法求 $[x]_{原}\div[y]_{原}$。

解析： $[|x|]_{补}=0.1001$　$[|y|]_{补}=0.1011$　$[-|y|]_{补}=1.0101$

	进位C_{out}	余数R	商Q	
		0.1001	0.0000	初始余数R_0是被除数x
$+[-\|y\|]_{补}$		1.0101		减去除数y
	0	1.1110	0.0000	$R_1<0$，上商0
		1.1100	0.000	R和Q一起左移一位
$+[\|y\|]_{补}$		0.1011		加上除数y
	1	0.0111	0.0001	$R_2>0$，上商1
		0.1110	0.001	R和Q一起左移一位
$+[-\|y\|]_{补}$		1.0101		减去除数y
	1	0.0011	0.0011	$R_3>0$，上商1
		0.0110	0.011	R和Q一起左移一位
$+[-\|y\|]_{补}$		1.0101		减去除数y
	0	1.1011	0.0110	$R_4<0$，上商0
		1.0110	0.110	R和Q一起左移一位
$+[\|y\|]_{补}$		0.1011		加上除数y
	1	0.0001	0.1101	$R_5>0$，上商1

商的符号 $=x$ 的符号 $\oplus$ y 的符号

$=1\oplus 0=1$

$[x]_{原}\div[y]_{原}=1.1101$

余数$R=0.0001\times 2^{-4}$

请同学们注意

❶ 最后一次运算上商时，若余数小于0，此时虽然所有上商位都已经得到了，但负余数还需要加上除数y恢复成正数。

❷ 不难发现，上商位与进位位的值是相同的。因此，在硬件逻辑实现时，可用加法器的进位输出作为上商的控制信号以及可控加减法电路的控制信号。

步骤

1. 初始化余数、商
2. 左移余数
3. 根据上一步余数符号选择 $+Y$ 或 $-Y$
4. 根据新余数符号上商
5. 重复 n 次
6. 若最后余数为负，加 Y 修正

易错

- 上商依据是“本步运算后的余数符号”，不是“上一步”
- 左移发生在加减之前
- 原码除法的商符仍由两数符号异或得到
- 余数符号应与被除数一致；不一致时需修正

知识关联：恢复余数法（3.15 定点数的除法运算——原码除法——恢复余数法）；补码除法（3.17 定点数的除法运算——补码的除法——不恢复余数法）；补码加减（3.4 定点数的加减运算——补码运算规则）